



School of Computer Science and Artificial Intelligence

Lab Assignment # 07

Course Title : **AI Assistant Coding**

Name of Student : **M. Sai Pallavi**

Enrollment No. : **2303A54063**

Batch No. **48**

Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Prompt: “explain the bug, and provide a corrected version that avoids this issue”

Expected Output:

Corrected function avoids shared list bug

Output Screenshot:

```

mutable_default_argument.py > ...
1  #Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix
2  # Bug: Mutable default argument
3  def add_item(item, items=[]):
4      items.append(item)
5      return items
6  print(add_item(1))
7  print(add_item(2))
8  # The above code will output:
9  # [1]
10 # [1, 2]
11 # This is because the default list is shared across all calls to the function.
12 # To fix this issue, we can use None as the default value and create a new list inside the fun
13 def add_item(item, items=None):
14     if items is None:
15         items = [] # Create a new list if items is None
16     items.append(item)
17     return items
18 # Test cases to verify the corrected function works properly
19 assert add_item(1) == [1] # Test case 1: Adding first item
20 assert add_item(2) == [2] # Test case 2: Adding second item, should not affect the first call
21 assert add_item(3, [0]) == [0, 3] # Test case 3: Adding item to a provided list, should not a
22 print("All test cases passed!")
23

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/Floating.py"
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/mutable_default_argument.py"
[1]
[1, 2]
All test cases passed!
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>

```

Ln 1, Col 3 Spaces: 4 UTF-8

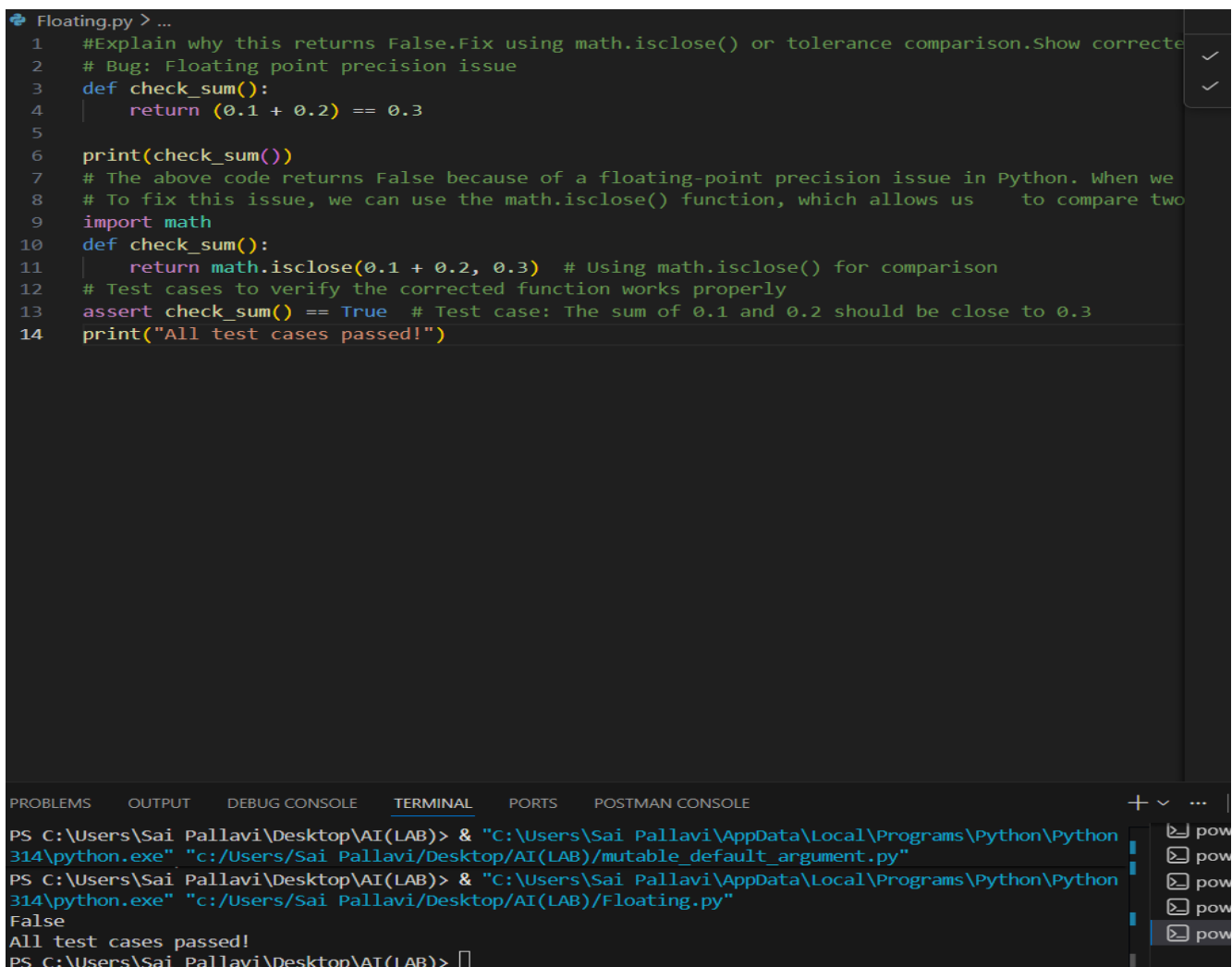
Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails. Use AI to correct with tolerance

prompt: Fix the if-condition that mistakenly uses = instead of ==, then run the corrected code with at least 3 assert test cases to confirm it works.

Expected Output:
Corrected function

Output Screenshot:



```
Floating.py > ...
1  #Explain why this returns False.Fix using math.isclose() or tolerance comparison.Show corrected code
2  # Bug: Floating point precision issue
3  def check_sum():
4      return (0.1 + 0.2) == 0.3
5
6  print(check_sum())
7  # The above code returns False because of a floating-point precision issue in Python. When we
8  # To fix this issue, we can use the math.isclose() function, which allows us to compare two
9  import math
10 def check_sum():
11     return math.isclose(0.1 + 0.2, 0.3) # Using math.isclose() for comparison
12 # Test cases to verify the corrected function works properly
13 assert check_sum() == True # Test case: The sum of 0.1 and 0.2 should be close to 0.3
14 print("All test cases passed!")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/mutable_default_argument.py"
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/Floating.py"
False
All test cases passed!
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>
```

Task Description #3 (Runtime Error – File Not Found)

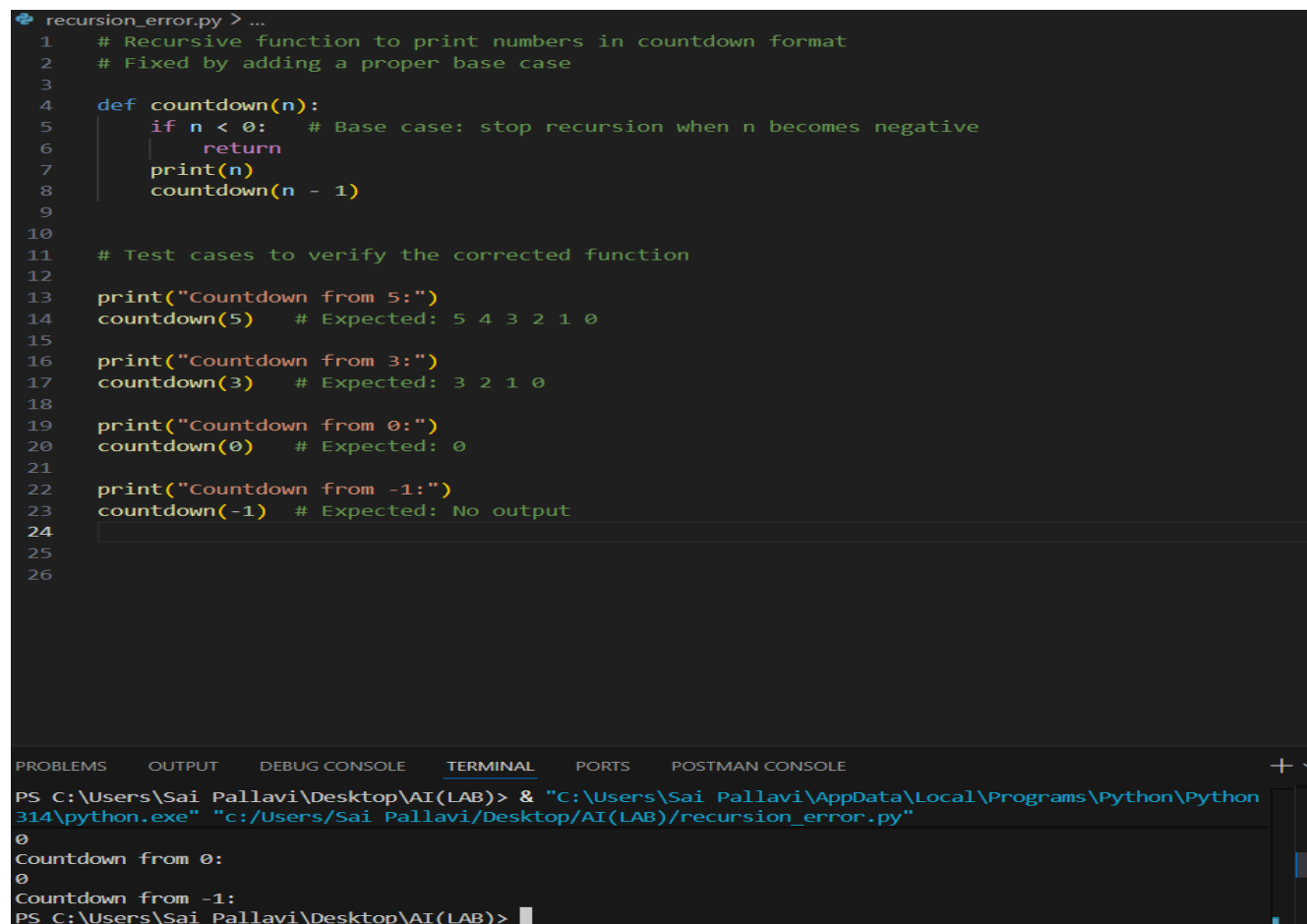
Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to Fix

Prompt: “explain the issue, and fix it by adding an appropriate stopping condition.”

Expected Output :

Correct recursion with stopping condition.

Output Screenshot:



```
recursion_error.py > ...
1  # Recursive function to print numbers in countdown format
2  # Fixed by adding a proper base case
3
4  def countdown(n):
5      if n < 0: # Base case: stop recursion when n becomes negative
6          return
7      print(n)
8      countdown(n - 1)
9
10
11 # Test cases to verify the corrected function
12
13 print("Countdown from 5:")
14 countdown(5) # Expected: 5 4 3 2 1 0
15
16 print("Countdown from 3:")
17 countdown(3) # Expected: 3 2 1 0
18
19 print("Countdown from 0:")
20 countdown(0) # Expected: 0
21
22 print("Countdown from -1:")
23 countdown(-1) # Expected: No output
24
25
26
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/recursion_error.py"
0
Countdown from 0:
0
Countdown from -1:
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>
```

Task 4 (Dictionary Key Error)

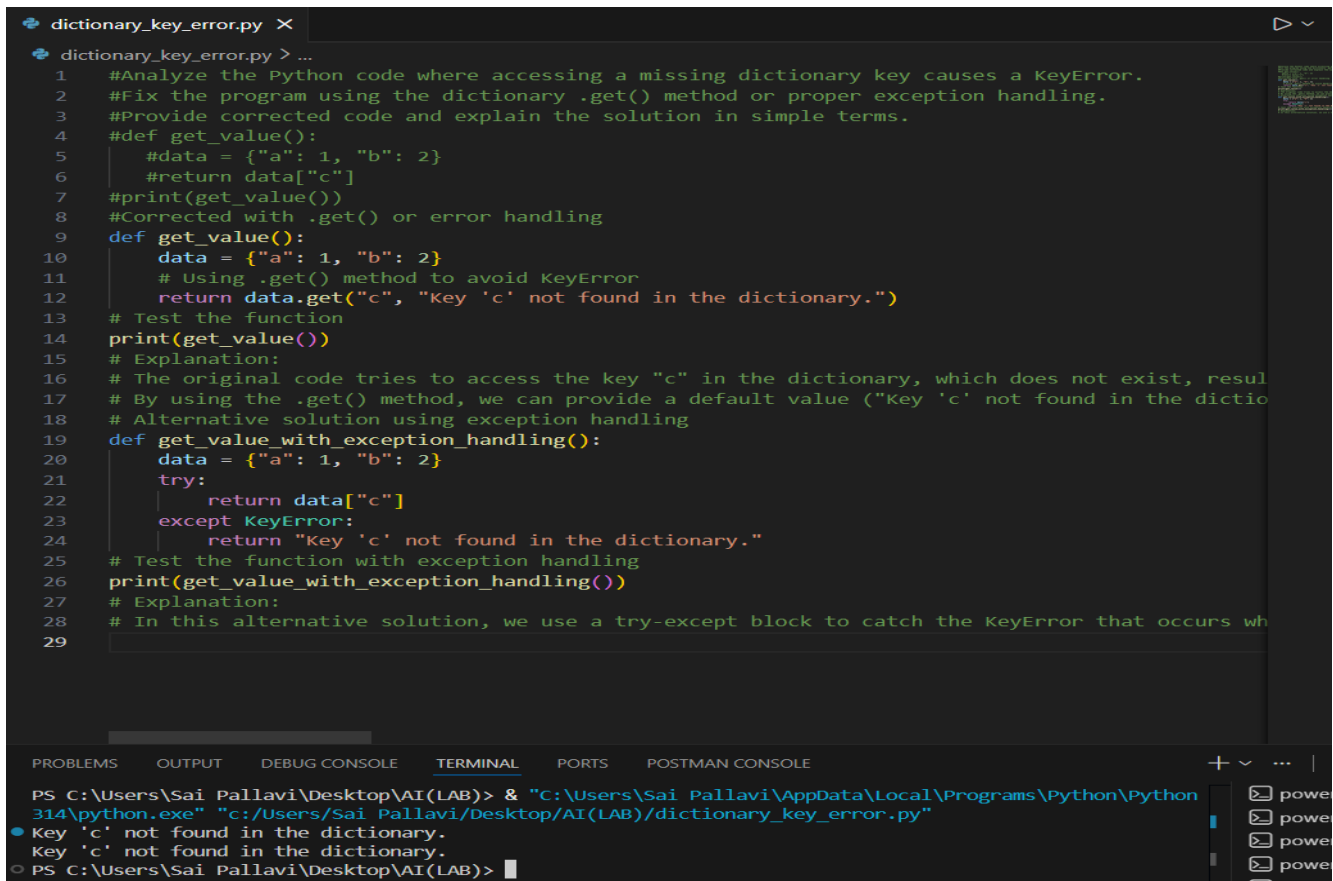
Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Prompt: “explain the issue, and fix it using .get() or proper error handling.”

Expected Output:

Corrected with .get() or error handling

Output Screenshot:



```
dictionary_key_error.py X
dictionary_key_error.py > ...
1  #Analyze the Python code where accessing a missing dictionary key causes a KeyError.
2  #Fix the program using the dictionary .get() method or proper exception handling.
3  #Provide corrected code and explain the solution in simple terms.
4  #def get_value():
5      #data = {"a": 1, "b": 2}
6      #return data["c"]
7  #print(get_value())
8  #Corrected with .get() or error handling
9  def get_value():
10     data = {"a": 1, "b": 2}
11     # Using .get() method to avoid KeyError
12     return data.get("c", "Key 'c' not found in the dictionary.")
13  # Test the function
14  print(get_value())
15  # Explanation:
16  # The original code tries to access the key "c" in the dictionary, which does not exist, result
17  # By using the .get() method, we can provide a default value ("Key 'c' not found in the dictio
18  # Alternative solution using exception handling
19  def get_value_with_exception_handling():
20     data = {"a": 1, "b": 2}
21     try:
22         return data["c"]
23     except KeyError:
24         return "Key 'c' not found in the dictionary."
25  # Test the function with exception handling
26  print(get_value_with_exception_handling())
27  # Explanation:
28  # In this alternative solution, we use a try-except block to catch the KeyError that occurs wh
29
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/dictionary_key_error.py"
● Key 'c' not found in the dictionary.
Key 'c' not found in the dictionary.
○ PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>
```

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Prompt: “explain why it never terminates, and fix it by updating the loop variable .”

Expected Output:

Corrected loop increments i

Output Screenshot:

```
infinite_loop.py > ...
1  #Analyze given code where loop never ends. Use AI to detect and fix it.
2  # Bug: Infinite loop
3  # def loop_example():
4  |   #i = 0
5  |   #while i < 5:
6  |   |   # print(i)
7  #Corrected loop increments i.
8  def loop_example():
9  |   i = 0
10 |   while i < 5:
11 |       print(i)
12 |       i += 1 # Incrementing i to avoid infinite loop
13 # Test cases to verify the corrected function works properly
14 loop_example() # Should print numbers from 0 to 4 and then stop
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/infinite_loop.py"
1
2
3
4
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> |
```

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Prompt: "fix the given code and explain the error"

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

wrong_unpacking.py > ...

```
1  #Analyze the Python code where tuple unpacking fails due to incorrect number of variables.
2  #Fix the error by correcting the unpacking or using '_' to ignore extra values.
3  #Provide corrected code, output, and explanation.
4  #a, b = (1, 2, 3)
5  # The above code will raise a ValueError because there are more values in the tuple than variables.
6  # To fix this issue, we can either correct the unpacking to match the number of values or use a wildcard.
7  # Corrected code with proper unpacking:
8  a, b, c = (1, 2, 3)
9  print(f"a: {a}, b: {b}, c: {c}")
10 # Output:
11 # a: 1, b: 2, c: 3
12 # Alternatively, if we only care about the first two values and want to ignore the rest:
13 a, b, _ = (1, 2, 3)
14 print(f"a: {a}, b: {b}")
15 # Output:
16 # a: 1, b: 2
17 # Explanation:
18 # In the original code, there were three values in the tuple (1, 2, 3) but only two variables (a, b).
19 # By correcting the unpacking to include a variable for each value (a, b, c), we can successfully unpack the tuple.
20 # Alternatively, if we only need the first two values and want to ignore the third, we can use a wildcard (_).
21
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/infinite_loop.py"

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/wrong_unpacking.py"

a: 1, b: 2, c: 3

a: 1, b: 2

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> █

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Prompt: Fix the given code and explain the error

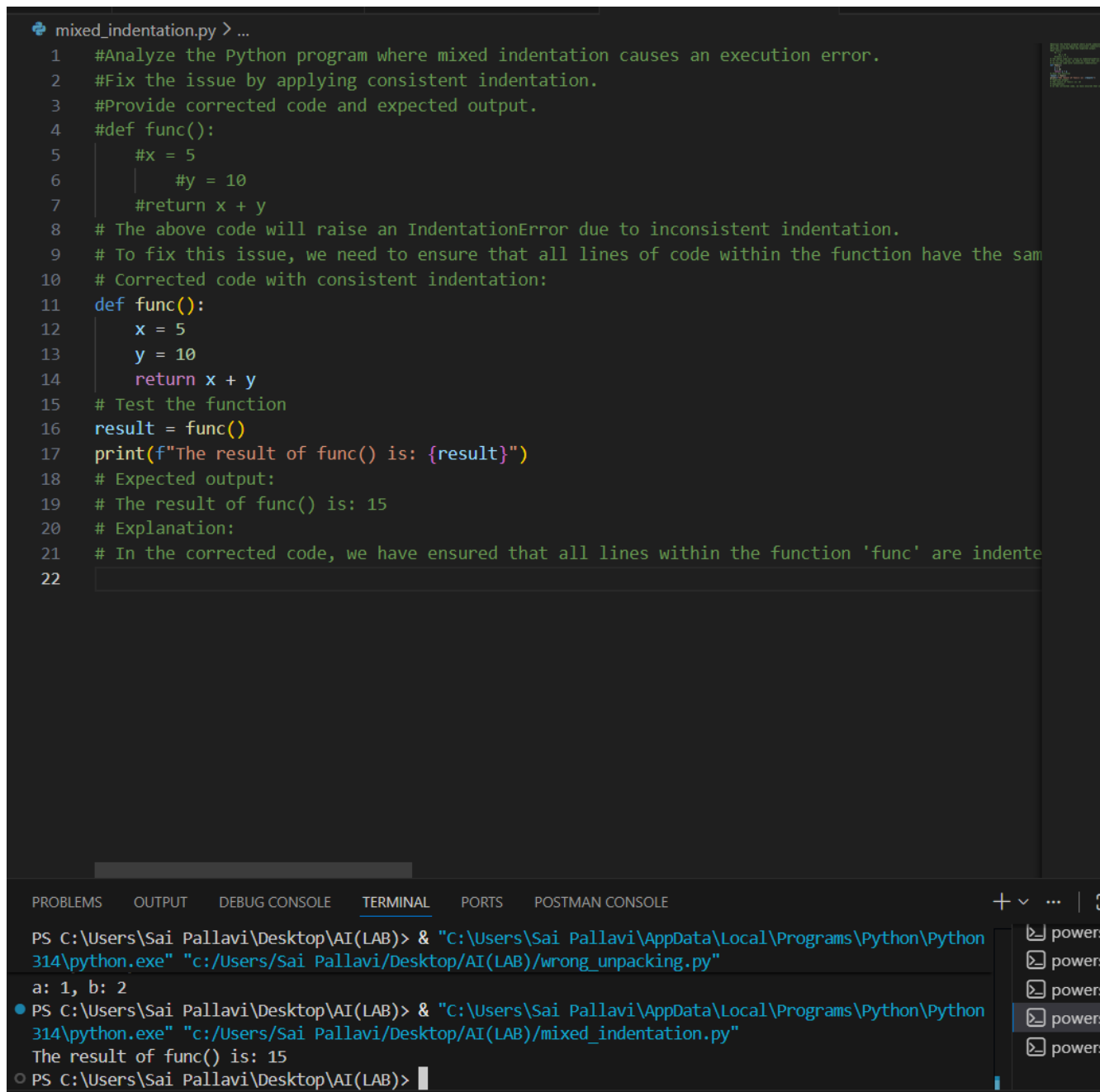
Bug: Mixed indentation

```
def func():  
    x = 5  
    y = 10  
    return x+y
```

Expected Output :

Consistent indentation applied.

Output Screenshot:



```
mixed_indentation.py > ...  
1  #Analyze the Python program where mixed indentation causes an execution error.  
2  #Fix the issue by applying consistent indentation.  
3  #Provide corrected code and expected output.  
4  #def func():  
5      #x = 5  
6      |   #y = 10  
7      |   #return x + y  
8  # The above code will raise an IndentationError due to inconsistent indentation.  
9  # To fix this issue, we need to ensure that all lines of code within the function have the same indentation.  
10 # Corrected code with consistent indentation:  
11 def func():  
12     x = 5  
13     y = 10  
14     return x + y  
15 # Test the function  
16 result = func()  
17 print(f"The result of func() is: {result}")  
18 # Expected output:  
19 # The result of func() is: 15  
20 # Explanation:  
21 # In the corrected code, we have ensured that all lines within the function 'func' are indented consistently.  
22
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/wrong_unpacking.py"  
a: 1, b: 2  
● PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/mixed_indentation.py"  
The result of func() is: 15  
○ PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>
```


Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

Prompt:fix the code and explain the error

```
# Bug: Wrong import
import maths
print(maths.sqrt(16))
```

Expected Output:
Corrected to import math

Output Screenshot:

```
wrong_import.py
1  #Analyze given code with incorrect import. Use AI to fix.
2  # Bug: Wrong import
3  #import maths
4  #print(maths.sqrt(16))
5  #orrected to import math
6  import math
7  print(math.sqrt(16)) # This will correctly print 4.0
8
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/wrong_import.py"
4.0
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
```

Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Prompt:fix the code and explain the error

```
# Bug: Early return inside loop
def total(numbers):
    for n in numbers:
        return n
print(total([1,2,3]))
```

Expected Output:

Corrected code accumulates sum and returns after loop

Output Screenshot:

```
return_inside_loop.py > ...
1  #Analyze the Python function where a return statement inside a loop causes early termination
2  #Fix the logic so the loop completes all iterations and returns the accumulated result.
3  #Provide corrected code, output, and explanation.
4  #def total(numbers):
5  |   #for n in numbers:
6  |       #return n
7  #print(total([1, 2, 3]))
8  # The above code will return 1 and exit the function immediately due to the return statement
9  # To fix this issue, we need to move the return statement outside the loop and accumulate th
10 def total(numbers):
11 |     total_sum = 0 # Initialize a variable to accumulate the total
12 |     for n in numbers:
13 |         total_sum += n # Add each number to the total_sum
14 |     return total_sum # Return the accumulated total after the loop completes
15 # Test the function with a list of numbers
16 result = total([1, 2, 3])
17 print(f"The total is: {result}")
18 # Output:
19 # The total is: 6
20 # Explanation:
21 # In the corrected code, we initialize a variable 'total_sum' to accumulate the sum of the n
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/return_inside_loop.py"
The total is: 6
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
```

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Prompt: fix the code and explain the error

Bug: Using undefined variable

```
def calculate_area():
    return length * width
print(calculate_area())
```

Requirements:

- Run the code to observe the error.

- **Ask AI to identify the missing variable definition.**
- **Fix the bug by defining length and width as parameters.**
- **Add 3 assert test cases for correctness.**

Expected Output :

- **Corrected code with parameters.**
- **AI explanation of the bug.**

Successful execution of assertions

Output Screenshot:

```
undefined_variable.py > ...
1  #Analyze given code where a variable is used before being defined. Let AI detect and fix the e
2  # Bug: Using undefined variable
3  def calculate_area():
4      |   return length * width
5  print(calculate_area())
6  # The above code will raise a NameError because 'length' and 'width' are not defined before th
7  # To fix this issue, we need to define 'length' and 'width' before using
8  # them in the function. We can either pass them as parameters to the function or define them a
9  # Fixing by passing parameters to the function:
10 def calculate_area(length, width):
11     |   return length * width
12 # Test cases to verify the corrected function works properly
13 assert calculate_area(5, 3) == 15 # Test case 1: Area of a rectangle with length 5 and width
14 assert calculate_area(7, 2) == 14 # Test case 2: Area of a rectangle with length 7 and width
15 print("All test cases passed!")
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/undefined_variable.py"
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/undefined_variable.py"
All test cases passed!
```

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Prompt: fix the code and explain the error

Bug: Adding integer and string

```
def add_values():
    return 5 + "10"
print(add_values())
```

Requirements:

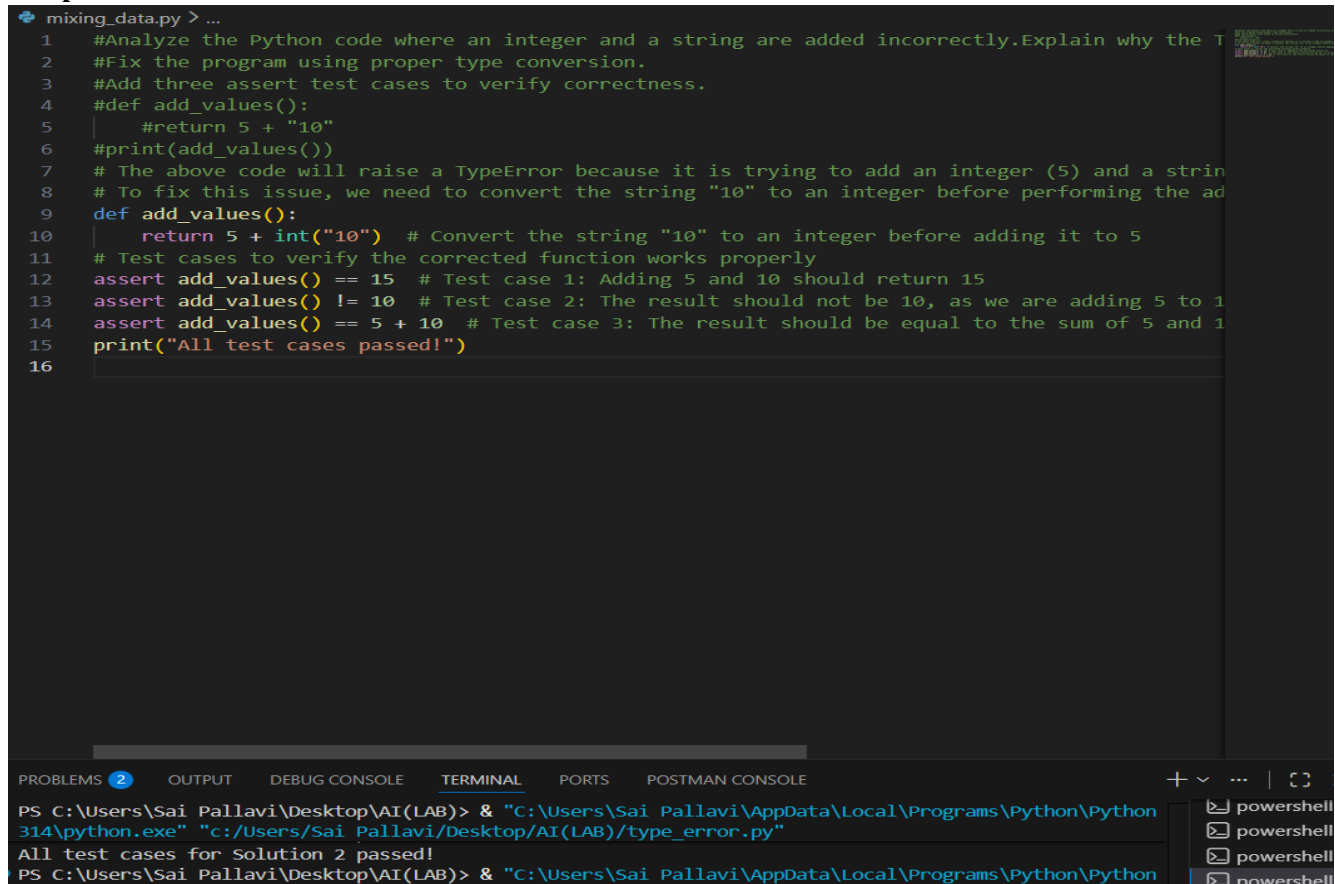
- Run the code to observe the error.
- AI should explain why `int + str` is invalid.
- Fix the code by type conversion (e.g., `int("10")` or `str(5)`).
- Verify with 3 assert cases.

Expected Output #6:

- **Corrected code with type handling.**
- **AI explanation of the fix.**

Successful test validation

Output Screenshot:



The screenshot shows a VS Code editor with a file named `mixing_data.py`. The code defines a function `add_values()` that returns `5 + "10"`. It includes three assert statements to verify the function's behavior: `assert add_values() == 15`, `assert add_values() != 10`, and `assert add_values() == 5 + 10`. The terminal output shows that all test cases passed.

```
1 #Analyze the Python code where an integer and a string are added incorrectly.Explain why the T
2 #Fix the program using proper type conversion.
3 #Add three assert test cases to verify correctness.
4 #def add_values():
5 |     #return 5 + "10"
6 #print(add_values())
7 # The above code will raise a TypeError because it is trying to add an integer (5) and a string
8 # To fix this issue, we need to convert the string "10" to an integer before performing the ad
9 def add_values():
10 |     return 5 + int("10") # Convert the string "10" to an integer before adding it to 5
11 # Test cases to verify the corrected function works properly
12 assert add_values() == 15 # Test case 1: Adding 5 and 10 should return 15
13 assert add_values() != 10 # Test case 2: The result should not be 10, as we are adding 5 to 1
14 assert add_values() == 5 + 10 # Test case 3: The result should be equal to the sum of 5 and 1
15 print("All test cases passed!")
16
```

Terminal Output:

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/type_error.py"
All test cases for Solution 2 passed!
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
```

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

Prompt: fix the error and explain the code

```
# Bug: Adding string and list
def combine():
    return "Numbers: " + [1, 2, 3]
print(combine())
```

Requirements:

- Run the code to observe the error.
- Explain why `str + list` is invalid.
- Fix using conversion (`str([1,2,3])` or `" ".join()`).
- Verify with 3 assert cases.

Expected Output:

- Corrected code

- Explanation
- Successful test validation

Output Screenshot:

```

1  #Analyze the Python code where a string is incorrectly concatenated with a list.Explain why th
2  #Fix the program using type conversion such as str() or join().
3  #Add three assert test cases to verify correctness.
4  #def combine():
5      #return "Numbers: " + [1, 2, 3]
6  #print(combine())
7  # The above code will raise a TypeError because it is trying to concatenate a string ("Numbers
8  # To fix this issue, we need to convert the list to a string before concatenating it with the
9  def combine():
10     return "Numbers: " + str([1, 2, 3]) # Convert the list to a string before concatenating
11 # Test cases to verify the corrected function works properly
12 assert combine() == "Numbers: [1, 2, 3]" # Test case 1: The result should be the string repre
13 assert combine() != "Numbers: 1, 2, 3" # Test case 2: The result should not be a string witho
14 assert combine() == "Numbers: " + str([1, 2, 3]) # Test case 3: The result should be equal to
15 print("All test cases passed!")
16
17

```

```

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/mixing_data.py"
All test cases passed!
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/string_list.py"
All test cases passed!

```

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

Prompt:fix the code and explain the error

Bug: Multiplying string by float

```

def repeat_text():
    return "Hello" * 2.5
print(repeat_text())

```

Requirements:

- Observe the error.
- Explain why float multiplication is invalid for strings.
- Fix by converting float to int.
- Add 3 assert test cases.

Output Screenshot:

```

multiplying_string.py > ...
1  #Detect the TypeError caused by multiplying a string with a floatExplain why the error occurs.
2  #Fix the code by converting the float to an integer.
3  #Add three assert test cases to validate the solution.
4  #def repeat_text():
5  |   #return "Hello" * 2.5
6  #print(repeat_text())
7  # The above code will raise a TypeError because it is trying to multiply a string ("Hello") by
8  # To fix this issue, we need to convert the float (2.5) to an integer before using it to repea
9  def repeat_text():
10 |   return "Hello" * int(2.5) # Convert the float to an integer before multiplying
11 # Test cases to verify the corrected function works properly
12 assert repeat_text() == "HelloHello" # Test case 1: The result should be "Hello" repeated 2 t
13 assert repeat_text() != "HelloHelloHello" # Test case 2: The result should not be "Hello" rep
14 assert repeat_text() == "Hello" * 2 # Test case 3: The result should be equal to "Hello" repe
15 print("All test cases passed!")
16
17
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/string_list.py"
All test cases passed!
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
```

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer.

Prompt:fix the code and explain the error

Bug: Adding None and integer

```
def compute():
    value = None
    return value + 10
print(compute())
```

Requirements:

- Run and identify the error.
- Explain why NoneType cannot be added.
- Fix by assigning a default value.

Validate using asserts

Output Screenshot:

```
adding_none.py > ...
1  #Analyze the Python code where None is added to an integer.Explain why NoneType causes a TypeE
2  #Fix the code by assigning a default value.
3  #Add assert statements to validate correctness.
4  #def compute():
5  |     #value = None
6  |     #return value + 10
7  #print(compute())
8  # The above code will raise a TypeError because it is trying to add None (which is of type Non
9  # To fix this issue, we can assign a default value to the variable 'value' instead of None. Fo
10 def compute():
11 |     value = 0 # Assign a default value instead of None
12 |     return value + 10 # Now we can safely add 10 to the default value
13 # Test cases to verify the corrected function works properly
14 assert compute() == 10 # Test case 1: The result should be 10
15 assert compute() != 0 # Test case 2: The result should not be 0, as we are adding 10 to the d
16 assert compute() == 0 + 10 # Test case 3: The result should be equal to the sum of the defaul
17 print("All test cases passed!")
18
19
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/multiplying_string.py"
All test cases passed!
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
```

Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

Prompt: fix the code and explain the error

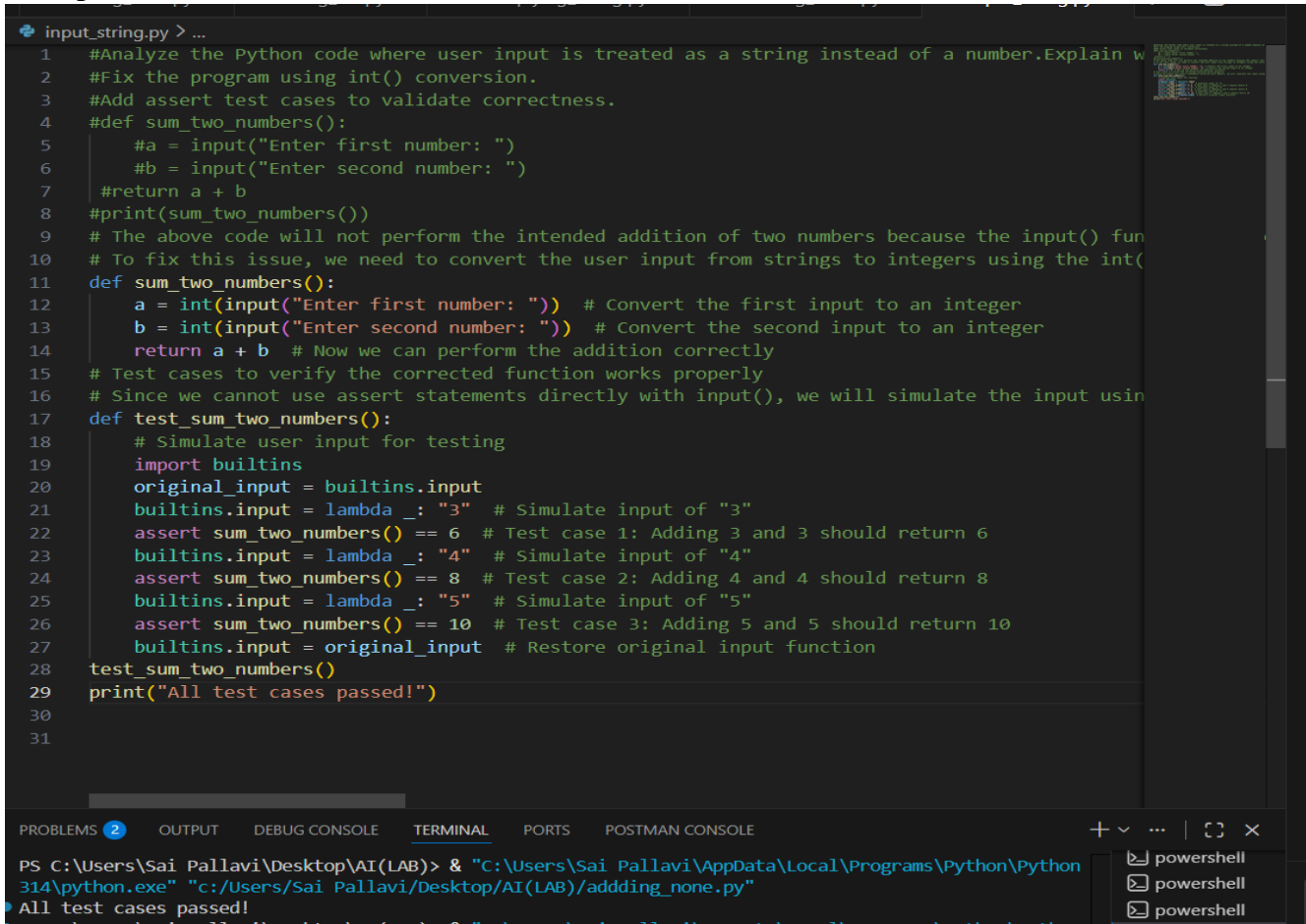
Bug: Input remains string

```
def sum_two_numbers():
    a = input("Enter first number: ")
    b = input("Enter second number: ")
    return a + b
print(sum_two_numbers())
```

Requirements:

- Explain why input is always string.
- Fix using int() conversion.
- Verify with assert test cases.

Output Screenshot:



```
input_string.py > ...
1 #Analyze the Python code where user input is treated as a string instead of a number.Explain w
2 #Fix the program using int() conversion.
3 #Add assert test cases to validate correctness.
4 #def sum_two_numbers():
5     #a = input("Enter first number: ")
6     #b = input("Enter second number: ")
7     #return a + b
8 #print(sum_two_numbers())
9 # The above code will not perform the intended addition of two numbers because the input() fun
10 # To fix this issue, we need to convert the user input from strings to integers using the int(
11 def sum_two_numbers():
12     a = int(input("Enter first number: ")) # Convert the first input to an integer
13     b = int(input("Enter second number: ")) # Convert the second input to an integer
14     return a + b # Now we can perform the addition correctly
15 # Test cases to verify the corrected function works properly
16 # Since we cannot use assert statements directly with input(), we will simulate the input usin
17 def test_sum_two_numbers():
18     # Simulate user input for testing
19     import builtins
20     original_input = builtins.input
21     builtins.input = lambda _: "3" # Simulate input of "3"
22     assert sum_two_numbers() == 6 # Test case 1: Adding 3 and 3 should return 6
23     builtins.input = lambda _: "4" # Simulate input of "4"
24     assert sum_two_numbers() == 8 # Test case 2: Adding 4 and 4 should return 8
25     builtins.input = lambda _: "5" # Simulate input of "5"
26     assert sum_two_numbers() == 10 # Test case 3: Adding 5 and 5 should return 10
27     builtins.input = original_input # Restore original input function
28 test_sum_two_numbers()
29 print("All test cases passed!")
30
31
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/adding_none.py"

• All test cases passed!

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/adding_none.py"

powerShell powerShell powerShell